

#include <stdio.h>

int main () {

int arr[100], n;

int smallest, second-smallest;

printf ("Enter number of elements: ");

scanf ("%d", &n);

printf ("Enter %d elements: \n", n);

for (int i = 0; i < n; i++) {

scanf ("%d", &arr[i]);

}

if (arr[0] < arr[1]) {

smallest = arr[0];

second-smallest = arr[1];

{ else {

smallest = arr[1];

second-smallest = arr[0];

}

for (int i = 2; i < n; i++) {

if (arr[i] < smallest) {

second-smallest = smallest;

smallest = arr[i];

{ elseif (arr[i] < second-smallest &

arr[i] != smallest) {

second-smallest = arr[i];

}

}

printf ("Second smallest element = %d \n", second-smallest);

O/P Enter number of element : 5

Enter 5 elements:

12

23

115

21

67

Second smallest element = 21.

Q2 Sum of left diagonal.

```
#include <stdio.h>
```

```
int main () {
```

```
int n, i, j;
```

```
int matrix[50][50];
```

```
int sum = 0;
```

```
printf ("Enter the size of the sq. matrix:");
```

```
scanf ("%d", &n);
```

```
printf ("Enter the elements of the %d x %d matrix: \n", n, n);
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < n; j++) {
```

```
scanf ("%d", &matrix[i][j]);
```

```
for (i = 0; i < n; i++) {
```

```
sum += matrix[i][i];
```

```
printf ("Sum of left diagonal = %d \n", sum);
```

```
return 0;
```

O/P

Enter the size of the square matrix: 2

Enter the elements of the 2x2 matrix:

2

3

3

4

Sum of left diagonal = 6.

Q1 Sumation Operation in matrix.

```
#include <stdio.h>

int main() {
    int rows, cols;
    int matrix[50][50];
    int i, j;

    printf("Enter number of rows:");
    scanf("%d", &rows);

    printf("Enter number of columns:");
    scanf("%d", &cols);

    printf("Enter the elements of the\n%d x %d matrix:\n", rows, cols);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    for (i = 0; i < rows; i++) {
        int row-sum = 0;
        for (j = 0; j < cols; j++) {
            row-sum += matrix[i][j];
        }
        printf("Sum of row %d = %d\n", i+1, row-sum);
    }

    for (j = 0; j < cols; j++) {
        int col-sum = 0;
```

```
for (i = 0; i < rows; i++) {
```

```
col-sum += matrix[i][j];
```

```
printf("Sum of column %d = %d\n",
```

```
j+1, col-sum);
```

```
return 0;
```

O/p

Enter number of rows: 2

Enter number of columns: 2

Enter the element of the 2x2 matrix:

23

32

45

32

Sum of row 1 = 58

Sum of row 2 = 77

Sum of column 1 = 68

Sum of column 2 = 64



Q4 Duplicate Numbers.

```
#include <stdio.h>
```

```
int main() {
    int arr[100], n;
    int i, j, count = 0;
    printf("Enter number of elements:");
    scanf("%d", &n);
    printf("Enter %d elements: \n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
                break;
            }
        }
    }
    printf("Total number of duplicate elements = %d \n", count);
    return 0;
}
```

O/p.

Enter number of elements: 4
Enter 4 elements:

2
3
4
5

Total number of duplicate elements = 0

Q3 2nd & 1st largest no.

```
#include <stdio.h>
```

```
int main() {
    int arr[100], n;
    int largest, second-largest;
    printf("Enter number of elements:");
    scanf("%d", &n);
    printf("Enter %d elements: \n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    if (arr[0] > arr[1]) {
        largest = arr[0];
        second-largest = arr[1];
    } else {
        largest = arr[1];
        second-largest = arr[0];
    }
    for (int i = 2; i < n; i++) {
        if (arr[i] > largest) {
            second-largest = largest;
            largest = arr[i];
        } else if (arr[i] > second-largest && arr[i] != largest) {
            second-largest = arr[i];
        }
    }
}
```



```
printf ("Second largest is %d\n",
second_largest);
```

```
printf ("Second largest element =
%d\n", second_largest);
return 0;
```

O/P

Enter number of element: 4

2

3

4

5

Second largest element = 4

LAB-1

Q. FCFS

~ #include <stdio.h>

struct process {

int pid;

int at;

int bt;

int ct;

int tat;

int wt;

int st;

};

int main() {

int n, i, j;

float avg_wt = 0, avg_tat = 0, avg_st = 0;

struct process p[100], temp;

printf ("Enter number of processes: ");

scanf ("%d", &n);

for (i = 0; i < n; i++) {

p[i].pid = i + 1;

printf ("Enter at and bt for P%d:",

p[i].pid);

scanf ("%d %d", &p[i].at, &p[i].bt);

for (i = 0; i < n - 1; i++) {

for (j = i + 1; j < n; j++) {

if (p[i].at > p[j].at) {

temp = p[i];

p[i] = p[j];




```
for i=0; i<n; i++) {  
printf("%d\t", i);  
printf(" P %d | +v.d | +v.d | +v.d | +v.d |  
+v.d | +v.d | +v.d | n",  
    p[i].pid;  
    p[i].at,  
    p[i].bt,  
    p[i].ct)  
    p[i].tot;  
    p[i].wt;  
}
```

Pid cut bt ct fat wt st
 P1 1 1 2 1 0 0
 avg. fat = 2.00
 avg. wt = 0.00
 avg. st = 0.00

Q) SJF

```
#include <stdio.h>
```

```
struct process
```

```
{  
    int pid, at, bt;  
    int ct, tat, wt, rt;  
    int completed;  
};
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0, idx;  
    float avg-tat = 0, avg-wt = 0, avg-rt = 0;
```

```
    struct process p[20];
```

```
    printf("Enter number of processes:");  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter at and bt for P%d:",  
               p[i].pid);
```

```
        scanf("%d %d", &p[i].at, &p[i].bt);
```

```
        p[i].completed = 0;
```

```
    }
```

```
    while (completed < n) {
```

```
        int min-bt = 9999;
```

```
        idx = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (p[i].at <= time || p[i].completed == 0)
```

```
            if (p[i].bt < min-bt) {  
                min-bt = p[i].bt;  
                idx = i;  
            }  
        }  
    }
```

```
    if (idx != -1) {
```

```
        time += p[idx].bt;
```

```
        p[idx].ct = time;
```

```
        p[idx].tat = p[idx].ct - p[idx].at;
```

```
        p[idx].wt = p[idx].tat - p[idx].bt;
```

```
        p[idx].rt = p[idx].wt;
```

```
        p[idx].completed = 1;
```

```
        completed++;
```

```
    }
```

```
    else {
```

```
        time++;
```

```
    }
```

```
}
```

```
    printf("\n pid \t at \t bt \t ct \t tat \t wt \t rt \t");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \n",  
               p[i].pid, p[i].at, p[i].bt, p[i].ct,  
               p[i].tat, p[i].wt, p[i].rt);
```

```
        avg-tat += p[i].tat;  
        avg-wt += p[i].wt;  
        avg-rt += p[i].rt;
```

```
    }
```

```
    avg-tat /= n;
```

```
    avg-wt /= n;
```

```
    avg-rt /= n;
```



```

printf ("In avg tat = %.2f", avg_tat/n);
printf ("In avg wt = %.2f", avg_wt/n);
printf ("In avg rt = %.2f", avg_rt/n);

return 0;
}

```

Q18

Enter no. of processes: 2

Enter at and bt for P1: 93

45

Enter at and bt for P2: 67

43

Pid at bt ct tat wt rt

P1 93 45 68 45 0 0

P2 67 63 111 44 1 1

avg tat = 44.50

avg wt = 0.50

avg rt = 0.50

2/13

LAB-2

Q. Write a C program to estimate scheduling algo to find TAT, WT priority.

code:

```
#include <stdio.h>
```

```
#define MAX 20
```

```
struct process {
```

```
int pid, at, bt, pr;
```

```
int ct, tat, wt, remaining;
```

```
};
```

```
void nonpreemptive(struct process p[], int n){
```

```
int time = 0, completed = 0, visited[MAX];
```

```
while (completed < n) {
```

```
int idx = -1;
```

```
int highest = 9999;
```

```
for (int i = 0; i < n; i++) {
```

```
if (p[i].at <= time && !visited[i]) {
```

```
if (p[i].pr < highest) {
```

```
highest = p[i].pr;
```

```
idx = i;
```

```
if (idx != -1) {
```

```
time += p[idx].bt;
```

```
p[idx].ct = time;
```

```
p[idx].tat = p[idx].ct - p[idx].at;
```




```
    p[i].wt = p[i].tst + -p[i].bt;
```

```
    visited[i] = 1;
```

```
    completed++;
```

```
    } else {
```

```
        time++;
```

```
    }
```

```
    printf("\n ---Non-preemptive Priority  
    Scheduling ---\n");
```

```
    printf("PID | AT | BT | PR | CT | TAT | WT |");
```

```
    float totalWT = 0; totalTAT = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d | %d | %d | %d | %d | %d |",
```

```
                p[i].pid, p[i].at, p[i].bt, p[i].pr,
```

```
                p[i].ct, p[i].tst, p[i].wt);
```

```
        totalWT += p[i].wt;
```

```
        totalTAT += p[i].tst;
```

```
    printf("Avg WT = %.2f \n", totalWT/n);
```

```
    printf("Avg TAT = %.2f \n", totalTAT/n);
```

```
void preemptive(struct p[10], int n) {
```

```
    int time = 0, completed = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].remaining = p[i].bt;
```

```
        while (completed < n) {
```

```
            int highest = 9999;
```

```
            for (int i = 0; i < n; i++) {
```

```
                if (p[i].at <= time && p[i].
```

```
                    remaining > 0) {
```

```
                    if (p[i].pr < highest) {
```

```
                        highest = p[i].pr;
```

```
                        idx = i;
```

```
            if (idx != -1) {
```

```
                p[idx].remaining--;
```

```
                time++;
```

```
                if (p[idx].remaining == 0) {
```

```
                    completed++;
```

```
                    p[idx].ct = time;
```

```
                    p[idx].tst = p[idx].ct - p[idx].at;
```

```
                    p[idx].wt = p[idx].tst + -p[idx].bt;
```

```
                } else {
```

```
                    time++;
```



--- 1400 - Preemptive Priority Scheduling ---

PID	AT	BT	PR	CT	TAT	WT
1	32	34	23	80	114	0
2	12	34	23	116	34	0

Avg WT = 7.00
Avg TAT = 41.00

--- Preemptive Priority Scheduling ---

PID	AT	BT	PR	CT	TAT	WT
1	32	34	23	66	34	0
2	12	34	23	80	88	34

Avg WT = 17.00
Avg TAT = 91.00

27/3

LAB-3

Round Robin:

```
#include <stdio.h>
```

```
int main() {
```

```
int n, timeQuantum;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
int burstTime[n], remainingTime[n];
```

```
int waitingTime[n], turnaroundTime[n];
```

```
int i, time = 0, done;
```

```
float totalWaitingTime = 0, totalTurnaroundTime = 0;
```

```
for (i = 0; i < n; i++) {
```

```
printf("Enter burst time for process %d: ", i+1);
```

```
scanf("%d", &burstTime[i]);
```

```
remainingTime[i] = burstTime[i];
```

```
printf("Enter time quantum:");
```

```
scanf("%d", &timeQuantum);
```

```
for (i = 0; i < n; i++) {
```

```
waitingTime[i] = 0;
```

```
do {
```

```
done = 1;
```

```
for (i = 0; i < n; i++) {
```

```
if (remainingTime[i] > 0)
```




```

done = 0;
if (remainingTime[i] > timeQuantum) {
    time = timeQuantum;
    remainingTime[i] = timeQuantum;
} else {
    time = remainingTime[i];
    waitingTime[i] = time - burstTime[i];
    remainingTime[i] = 0;
}
}

while (!done) {
    for (t = 0; t < n; t++) {
        turnaroundTime[i] = burstTime[i] +
            waitingTime[i];
        totalWaitingTime += waitingTime[i];
        totalTurnaroundTime += turnaroundTime[i];
    }

    printf("\n Process | t Burst Time | t waiting
        Time | t Turnaround Time \n");
    for (t = 0; t < n; t++) {
        printf("P %d | t %d | t %d | t %d | t %d\n",
            t+1, burstTime[i], waitingTime[i],
            turnaroundTime[i], totalWaitingTime);
    }
}

```

```

printf("\n Avg. waiting time = %.2f, total
    waitingTime | n);
printf("\n Avg. Turn around Time = %.2f\n",
    totalTurnaroundTime | n);
return 0;
}

```

O/P

```

Enter number of processes: 4.
Enter burst time for process 1: 34
Enter burst time for process 2: 56
Enter burst time for process 3: 78
Enter burst time for process 4: 89
Enter time quantum: 23

```

Process	Burst Time	Waiting Time	Turnaround Time
P ₁	34	69	103
P ₂	56	126	182
P ₃	78	159	237
P ₄	89	168	257

Avg. waiting time = 130.50

Avg. turnaround time = 194.75

C/P

Enter number of processes: 4
 Enter burst time for process 1: 34
 Enter burst time for process 2: 36
 " " " " " 3: 78
 " " " " " 4: 89

Enter time Quantum: 13.

Process	Burst time	Waiting time	Turnaround time
P ₁	34	78	112
P ₂	36	138	194
P ₃	78	155	233
P ₄	89	168	257

Avg. Waiting time = 134.75

Avg. Turnaround time = 199.00

Quantum time: 23 is more optimal.

Q. Multilevel queue:

```
#include <stdio.h>
#include <string.h>
#define MAX 10
```

```
struct process {
    int pid;
    int arrival;
    int burst;
    int remaining;
    char type[100];
};
```

```
struct queue {
    struct process data[MAX];
    int front, rear;
};
```

```
void initQueue (struct queue *q) {
    q->front = q->rear = -1;
}

int isEmpty (struct queue *q) {
    return q->front == -1;
}
```

```
void enqueue (struct queue *q, struct process p) {
    if (q->rear == MAX-1) {
        return;
    }
    if (q->front == -1) {
        q->front = 0;
    }
    q->data[q->rear] = p;
    q->rear++;
}
```



```

struct process enqueue (struct queue *q)
{
    struct process p = q->data[q->front];
    if (q->front == q->rear)
        q->front = q->rear = -1;
    else
        q->front++;
}

```

return p;

```

void sortByArrival (struct process p[], int n)
{
    int i, j;
    struct process temp;
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[i].arrival > p[j].arrival)
            {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
}

```

```

int main()
{
    int n, i, completed = 0, current_time = 0;
    struct process p[10];
    struct queue systemQueue, userQueue;
}

```

```

struct process current-process;
int hasCurrent = 0;
initQueue (&systemQueue);
initQueue (&userQueue);

printf ("Enter no. of Processes: ");
scanf ("%d", &n);

for (int k = 0; k < n; k++)
{
    printf ("\n Enter Process ID: ");
    scanf ("%d", &p[k].pid);
    printf ("Enter Arrival Time: ");
    scanf ("%d", &p[k].arrival);
    printf ("Enter Burst Time: ");
    scanf ("%d", &p[k].burst);
    printf ("Enter Type (System/user): ");
    scanf ("%s", p[k].type);
}

sortByArrival (p, n);

while (completed < n)
{
    while (i < n && p[i].arrival <= current_time)
    {
        if (strcmp (p[i].type, "system") == 0)
            enqueue (&systemQueue, p[i]);
        else
            enqueue (&userQueue, p[i]);
        i++;
    }
}

```

Process 2

Enter at: 1

Enter bt: 4

Enter type (0 = system, 1 = user): 0

Process 3

Enter at: 3

Enter bt: 2

Enter type (0 = system, 1 = user): 1

ID	Type	AT	BT	CT	WT	TAT
0	system	0	5	5	0	5
1	user	2	3	12	7	10
2	system	1	4	9	4	8
3	user	3	2	14	9	11

Avg. WT: 5.00

Avg. Turnaround Time: 8.50

8/2
27/3

10 Write a program to simulate Real time CPU scheduling algo.

- Rate Monotonic
- Earliest deadline
- Proportional scheduling

```
#include <stdio.h>
#include <math.h>
#define MAX 10

int bt[MAX], d[MAX], p[MAX], id[MAX];
int n;

void EDF() {
    int bt2[MAX], d2[MAX], id2[MAX];
    int ct[MAX], wt[MAX], tat[MAX];

    for (int i = 0; i < n; i++) {
        bt2[i] = bt[i];
        d2[i] = d[i];
        id2[i] = id[i];
    }

    float u = 0;
    for (int i = 0; i < n; i++) {
        u += (float) bt[i] / p[i];
    }

    printf("\n -- EDF Scheduling -- \n");
    printf("CPU Utilization: %.2f \n", u);
    if (u > 1) {
        printf("Not Schedulable \n");
        return;
    }
}
```




```

for (int i = 0; i < n - 1; i++)
    for (int j = i + 1; j < n; j++)
        if (d2[i] > d2[j]) {
            int t;
            t = d2[i]; d2[i] = d2[j]; d2[j] = t;
            t = b2[i]; b2[i] = b2[j]; b2[j] = t;
            t = id2[i]; id2[i] = id2[j]; id2[j] = t;
        }

```

```

int time = 0;
for (int i = 0; i < n; i++) {
    time += b2[i];
    ct[i] = time;
    tat[i] = ct[i];
    wt[i] = tat[i] - b2[i];
}
printf("SD BT D ct WT TAT\n");
for (int i = 0; i < n; i++)
    printf("%d %d %d %d %d %d\n",
           id2[i], b2[i], d2[i], ct[i], wt[i],
           tat[i]);

```

```

void rms() {
    int b2[MAX], p2[MAX], id2[MAX];
    int ct[MAX], wt[MAX], tat[MAX];
    for (int i = 0; i < n; i++) {
        b2[i] = b[i];
        p2[i] = p[i];
        id2[i] = id[i];
    }
}

```

```

float u = 0;
for (int i = 0; i < n; i++)
    u += (float) b2[i] / p2[i];

float bound = n * (pow(2, 1.0/n) - 1);
printf("\n-- RMS scheduling --\n");
printf("CPU utilization: %.2f\n", u);
printf("Bound: %.3f\n", bound);

if (u > bound)
    printf("not schedulable\n");
return;

```

```

for (int i = 0; i < n - 1; i++)
    for (int j = i + 1; j < n; j++)
        if (p2[i] > p2[j]) {
            int t;
            t = p2[i]; p2[i] = p2[j]; p2[j] = t;
            t = b2[i]; b2[i] = b2[j]; b2[j] = t;
            t = id2[i]; id2[i] = id2[j]; id2[j] = t;
        }

```



```

int time = 0;
for (int i = 0; i < n; i++) {
    time += b[i];
    ct[i] = time;
    tat[i] = ct[i];
    wt[i] = tat[i] - b[i];
    printf("ID BT CT WT TAT\n");
    for (int i = 0; i < n; i++)
        printf("%d %d %d %d %d\n",
            id[i], b[i], ct[i], wt[i], tat[i]);
}

```

```

void proportional() {
    int total-w = 0, t = 0, total-time;
    for (int i = 0; i < n; i++)
        total-w += w[i];
    printf("Enter time: ");
    scanf("%d", &total-time);
    printf("In -- Proportional Scheduling.\n");
    while (t < total-time) {
        for (int i = 0; i < n; i++) {
            int slice = (w[i] * total-time) /
                total-w;

```

```

for (int j = 0; j < slice && total-time;
    j++)
    {
        for (int i = 0; i < S; i++)
        printf("Time %d, Process %d\n",
            t, id[i]);
        t++;
    }
}

int main() {
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        id[i] = i;
        printf("Process %d\n", i);
        printf("Burst time: ");
        scanf("%d", &b[i]);
        printf("Deadline: ");
        scanf("%d", &d[i]);
        printf("Period: ");
        scanf("%d", &p[i]);
        printf("Weight: ");
        scanf("%d", &w[i]);
    }
}

```

endf();

rms();

proportional();

return 0;

S/P

Enter no. of process : 3

Process : 0

Burst time: 3

Deadline: 7

Period: 20

Weight: 5

Process: 1

Burst Time: 2

Deadline: 4

Period: 5

Weight: 7

Process: 2

BT: 2

Deadline: 8

Period: 10

Weight: 23

CPU Utilization: 0.78

ID	BT	D	CT	WT	TAT
1	2	4	2	0	2
0	3	7	3	2	5
2	2	8	7	3	7

-- RMS Scheduling

CPU Utilization: 0.78

Bound: 0.780

ID	BT	P	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

S/P
only



Q Bounded buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define size 5
int buffer[size];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;
```

```
void *producer(void *arg) {
    for (int i = 0; i < 15; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Produced: %d at buffer [%d]\n", i, in);
        in = (in + 1) % size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}
```

```
void *consumer(void *arg) {
    for (int i = 0; i < 15; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from buffer [%d]\n", item, out);
        out = (out + 1) % size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(150000);
    }
    return NULL;
}
```

```
int main() {
    pthread_t p, c;
    sem_init(&empty, 0, size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```


produced: 0 at buffer[0]
consumed: 0 from buffer[1].
produced: 1 at buffer[1]
~~produced: 1~~
consumed: 1 from buffer[1]

consumed 0.13 from buffer [3]
consumed 0.14 from buffer [4]

Q Dining Philosopher

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t forks[4];

void * philosopher (void * num) {
    int id = (int *) num;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        int test = id;
        int right = (id+1) % N;
        if (test % 2 == 0) {
```

```

sem_wait(&forks[right]);
printf("Philosopher %d picked up the\n", id, right);

sem_wait(&forks[left]);
printf("Philosopher %d picked up the\n", id, left);
}
else {
sem_wait(&forks[left]);
printf("Philosopher %d picked up the\n", id, left);

sem_wait(&forks[right]);
printf("Philosopher %d picked up the\n", id, right);
}
printf("Philosopher %d is eating.\n", id);
sleep(2);

sem_post(&forks[left]);
sem_post(&forks[right]);
printf("%d, %d, %d, id, left, right

```

```
int main() {
    pthread_t pt1;
    int id1;

```



```
for (int i=0; i<N; i++)
```

```
sem_init (&mutex[i], 0, 1);
```

```
for (int i=0; i<N; i++)
```

```
{
```

```
ids[i] = i;
```

```
pthread_create (&ph[i], NULL, philosopher, &ids[i]);
```

```
}
```

```
for (int i=0; i<N; i++)
```

```
pthread_join (ph[i], NULL);
```

```
return 0;
```

```
}
```

O/P

Philosopher 0 is thinking

Philosopher 3 is thinking

Philosopher 1 is thinking

Philosopher 4 is thinking

Philosopher 2 is thinking

Philosopher 2 is picked up left fork

Philosopher 1 is picked up right fork

Trace: 1

buffer = [-, -, -, -, -]

in = 0, out = 0

empty = 3, full = 0

0th philosopher: 0 at buffer[0]

buffer = [0, -, -, -, -]

in = 1, out = 0

empty = 4, full = 1

1st philosopher: 0 at buffer[1]

buffer = [0, 1, -, -, -]

in = 1, out = 1

empty = 5, full = 0

Trace: 2

forks = [1, 1, 1, 1, 1]

All thinking

P1 picked up left fork

P1 picked up right fork

P1 is eating

forks = [1, 0, 0, 1, 1]



Lab # 05

Q. Banker's algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
#define MAX10
int n, m;
int allocation [max][max];
int max [max][max];
int need [max][max];
int available [max];

void calculateNeed () {
    int i, j;
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            need [i][j] = max [i][j] - allocation [i][j];
        }
    }
}

int SafetyAlgorithm () {
    int work [max];
    int finish [max];
    int SafeSequence [max];

    int i, j, k;
    for (int i=0; i<m; i++) {
        work [i] = available [i];
    }
    for (int i=0; i<n; i++) {
```

```
int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish [i] == 0) {
            for (j=0; j<m; j++) {
                if (need [i][j] > work [j])
                    break;
            }
            if (j == m) {
                for (k=0; k<m; k++) {
                    work [k] += allocation [i][k];
                }
                SafeSequence [count++] = i;
                finish [i] = 1;
                found = 1;
            }
        }
    }
    if (found == 0)
        break;
}

if (count == 0) {
    printf ("\n System is in safe state\n");
    printf ("Safe Sequence:");
    for (i=0; i<n; i++) {
        printf ("%d ", SafeSequence [i]);
    }
}
```

```

    if (i != n-1)
        printf("-");
    }
    printf("\n");
    return 1;
}
else {
    printf("\n System is in unsafe state\n");
    return 0;
}
}

```

```

void ResourceRequest() {
    int process;
    int Request[100];
    int i;
    printf("\n Enter Process Number: ");
    scanf("%d", &process);
    printf("Enter Request Vector:\n");
    for (i = 0; i < m; i++) {
        scanf("%d", &Request[i]);
    }
    for (i = 0; i < m; i++) {
        if (Request[i] > Need[process][i])
            printf("\n Error. Request exceeds\n");
        else {
            for (i = 0; i < m; i++) {
                if (Request[i] > Available[i]) {
                    printf("\n Resources not available");
                    return;
                }
                Available[i] -= Request[i];
                Allocation[process][i] += Request[i];
                Need[process][i] -= Request[i];
            }
            printf("\n checking for Safe State\n");
            if (SafetyAlgorithm()) {
                printf("\n request can be granted\n");
            }
            else {
                for (i = 0; i < m; i++) {
                    Available[i] += Request[i];
                    Allocation[process][i] -= Request[i];
                    Need[process][i] += Request[i];
                }
                printf("\n request can't be granted");
            }
        }
    }
}

```

```

for (i = 0; i < m; i++) {
    if (Request[i] > Available[i]) {
        printf("\n Resources not available");
        return;
    }
    Available[i] -= Request[i];
    Allocation[process][i] += Request[i];
    Need[process][i] -= Request[i];
}
printf("\n checking for Safe State\n");
if (SafetyAlgorithm()) {
    printf("\n request can be granted\n");
}
else {
    for (i = 0; i < m; i++) {
        Available[i] += Request[i];
        Allocation[process][i] -= Request[i];
        Need[process][i] += Request[i];
    }
    printf("\n request can't be granted");
}
}

```

```

}
else {
    for (i = 0; i < m; i++) {
        Available[i] += Request[i];
        Allocation[process][i] -= Request[i];
        Need[process][i] += Request[i];
    }
    printf("\n request can't be granted");
}
}
}

```

```

int main() {
    int t, j;
    int choice;
    printf("Enter no. of process:");
    scanf("%d", &n);

    printf("Enter no. of resource types:");
    scanf("%d", &m);

    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            scanf("%d", &MAX[i][j]);
        }
    }

    printf("Enter available resources:");
    for (i=0; i<m; i++) {
        scanf("%d", &Available[i]);
    }

    calculateNeed();

    do {
        printf("1. Safety Algo");
        printf("2. Resource Request");
        printf("3. Exit");

        printf("\nEnter choice:");
        scanf("%d", &choice);
    } while (choice != 3);
}

```

Ans.

Q1P Enter no. of Processes: 5
 Enter no. of resource: 3
 Enter allocation matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter max. matrix:

7	5	3
3	2	2
4	0	2
2	2	2
4	3	3

Enter available resource:

3	3	2
---	---	---

Enter choice: 1
 System is in Safe State
 Safe Sequence :- P1 → P3 → P4 → P0 → P2.

Enter choice: 2
 Enter process no: 3
 Enter request vector:

3	3	4
---	---	---

Process exceed max. claim.

0. Deadlock detection.

```
#include <stdio.h>
#define MAX 10

int main()
{
    int n, m;
    int allocation[MAX][MAX];
    int request[MAX][MAX];
    int available[MAX];
    int work[MAX];
    int finish[MAX];
    int i, j, k;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resources: ");
    scanf("%d", &m);

    printf("Enter allocation matrix: \n");
    scanf("%d", &m);

    printf("Enter request matrix: \n");
    for (i=0; i<n; i++)
    {
        for (j=0; j<m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter request matrix: \n");
    for (i=0; i<n; i++)
    {
        for (j=0; j<m; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }
}
```

```
printf("\nEnter Available resources: \n");
for (i=0; i<m; i++)
{
    scanf("%d", &available[i]);
}

for (i=0; i<n; i++)
{
    int allzero = 1;
    for (j=0; j<m; j++)
    {
        if (allocation[i][j] != 0)
        {
            allzero = 0;
            break;
        }
    }
    if (allzero)
    {
        finish[i] = 1;
    }
    else
    {
        finish[i] = 0;
    }
}

int found;
do
{
    found = 0;
    for (i=0; i<n; i++)
    {
        if (finish[i] == 0)
        {
            for (j=0; j<m; j++)
            {
                if (request[i][j] > work[j])
                {
                    break;
                }
            }
            if (j == m)
            {
                for (k=0; k<m; k++)
                {
                    work[k] = work[k] + allocation[i][k];
                }
                finish[i] = 1;
                found = 1;
            }
        }
    }
} while (found == 0);

if (found == 0)
{
    printf("System is in deadlock state\n");
}
else
{
    printf("System is not in deadlock state\n");
}
```

Work + Allocation Matrix;

finish[i] = 1;

found = 1;

}

}

while(found)

int deadlock = 0;

printf("No Deadlocked Processes:\n");

for (i = 0; i < n; i++)

if (finish[i] == 0)

printf("P%d\n", i);

deadlock = 1;

}

if (deadlock == 0)

printf("No deadlock detected\n");

return 0;

}

O/p.

Enter no. of process: 3

Enter no. of resource type: 3

Enter allocation matrix;

2 3 4

2 1 2

3 1 2

Enter request matrix;

3 3 1

2 1 2

2 1 2

Enter available resources;

1 0 3

Deadlocked processes:

P0 P1 P2

if (i == 0) {

if (i == 1) {

if (i == 2) {

if (i == 3) {

if (i == 4) {

if (i == 5) {

if (i == 6) {

if (i == 7) {

if (i == 8) {

if (i == 9) {

if (i == 10) {

if (i == 11) {

if (i == 12) {

if (i == 13) {

if (i == 14) {

if (i == 15) {

if (i == 16) {

if (i == 17) {



Q. Write a C program to simulate the following contiguous memory allocation techniques.

- worst fit
- best fit
- first-fit.

```
#include <stdio.h>

void firstfit (int blockSize[], int blocks,
              int processSize[], int processes)

int allocation [processes];

for (int i=0; i<processes; i++) {
    allocation[i] = -1;
}

for (int i=0; i<processes; i++) {
    for (int j=0; j<blocks; j++) {
        if (blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] = 0;
            break;
        }
    }

    printf("\n First fit allocation : \n");
    printf("Process no. \t Process size \t\n");
    printf("block no. \n");
    for (int i=0; i<processes; i++) {
        printf("%d \t %d \t", i,
```

```

    }
    else {
        printf("Not allocated \n");
    }
}

void bestfit (int blockSize[], int blocks,
             int processSize[], int processes) {
    int allocation [processes];

    for (int i=0; i<processes; i++) {
        allocation[i] = -1;
    }

    for (int i=0; i<processes; i++) {
        int bestIdx = -1;

        for (int j=0; j<blocks; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] <
                    blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }

            if (bestIdx != -1) {
                allocation[i] = bestIdx;
                blockSize[bestIdx] = 0;
            }
        }
    }
}

```

```

printf("\n Best fit allocation : \n");
printf("Process no. | Process Size | Block  
no. | \n");
for (int i = 0; i < processes; i++) {
    printf("%d | %d | %d | %d", i+1, process[i].size,
           allocation[i], process[i].size);
    if (allocation[i] == -1) {
        printf("\n");
        printf("Not allocated \n");
    }
}

void worstfit (int block size[], int blocks,
               int process size[], int processes) {
    int allocation[processes];
    for (int i = 0; i < processes; i++) {
        allocation[i] = -1;
        for (int j = 0; j < blocks; j++) {
            if (block size[j] >= process size[i]) {
                if (worstidx == -1 || block size[j] > block size[worstidx]) {
                    worstidx = j;
                }
            }
        }
        if (worstidx != -1) {
            allocation[i] = block size[worstidx];
        }
    }
}

```

```

if (worstidx != -1) {
    allocation[i] = block size[worstidx];
    block size[worstidx] = 0;
}

printf("worst allocation");
for (int i = 0; i < processes; i++) {
    printf("%d | %d | %d | %d", i+1,
           process size[i],
           allocation[i], process size[i]);
    if (allocation[i] != -1) {
        printf("\n");
    } else {
        printf("Not allocated \n");
    }
}

int main() {
    int blocks, processes;
    printf("Enter no. of memory blocks: ");
    for (int i = 0; i < blocks; i++) {
        scanf("%d", &block size[i]);
    }
    printf("Enter no. of processes: ");
    for (int i = 0; i < processes; i++) {
        scanf("%d", &process size[i]);
    }
}

```



```

printf("Enter no. of memory blocks: ");
for (int i=0; i<processes; i++) {
    scanf("%d", &processSize[i]);
}

int block1[blocks], block2[blocks], block3[blocks];
for (int i=0; i<blocks; i++) {
    for (int i=0; i<blocks; i++)
    block1[i] = blocksSize[i];
    block2[i] = 0;
    block3[i] = 0;
}

first fit (block1, blocks, processSize,
processes);
best fit ( " )
worst fit ( " )
return 0;
}

```

%e

Enter no. of memory blocks: 6
Enter sizes of memory blocks:
300 600 350 200 750 125
Enter no. of processes: 5
Enter size of processes
115 500 358 200 375

First fit Allocation

process no.	process size	Block no.
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not allocated

process no.	process size	Block no.
1	115	6
2	500	2
3	358	5
4	200	4
5	375	NA

process no.	process size	Block no.
1	115	5
2	500	2
3	358	NA
4	200	3
5	375	NA

OK

Q write a program to simulate Page replacement algorithm

- a) FIFO
- b) LRU
- c) Optimal

sol #include <stdio.h>

```
int Search (int key, int frame[], int f){
    for (int i=0; i<f; i++) {
        if (frame[i] == key)
            return i;
    }
    return -1;
}
```

```
void FIFO (int Pages[], int n, int f){
    int frame[f], i, j, index = 0, faults = 0;
    for (i=0; i<n; i++)
        frame[i] = -1;
    for (i=0; i<n; i++) {
        if (!Search (Pages[i], frame, f)) {
            frame[index] = Pages[i];
            index = (index + 1) % f;
            faults++;
        }
    }
```

```
printf ("Page id -> ", Pages[i]);
for (j=0; j<f; j++)
    printf ("%d", frame[j]);
printf ("/n");
}
```

```
printf ("Total Page faults = %d\n", faults);
```

```
void LRU (int Pages[], int n, int f){
    int frame[f], time[f];
    int i, j, pos, faults = 0, counter = 0;
    for (i=0; i<n; i++) {
        frame[i] = -1;
        time[i] = 0;
        for (j=0; j<n; j++) {
            int found = 0;
            for (l=0; l<f; l++) {
                if (frame[l] == Pages[i]) {
                    counter++;
                    time[l] = counter;
                    found = 1;
                    break;
                }
            }
            if (!found) {
                int min = time[0];
                pos = 0;
                for (j=1; j<f; j++) {
                    if (time[j] < min) {
                        min = time[j];
                        pos = j;
                    }
                }
                frame[pos] = Pages[i];
                counter++;
                time[pos] = counter;
                faults++;
            }
        }
    }
```

```
printf ("Total Page faults = %d\n", faults);
```



```

for (j=0; j<f; j++)
    printf ("%d", frame[j]);
printf ("\n");
}
printf ("Total Page Faults = %d\n", faults);

void optimal (int pages[], int n, int f) {
    int frame [f];
    int i, j, k, faults = 0;
    for (i=0; i<f; i++)
        frame[i] = -1;
    for (i=0; i<n; i++) {
        if (!Search (pages[i], frame, f)) {
            int pos = -1, farthest = -1;
            for (j=0; j<f; j++) {
                if (frame[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if (pos == -1) {
                int max = -1;
                for (j=0; j<f; j++) {
                    if (found == 0) {
                        for (k=i+1; k<n; k++) {
                            if (frame[j] == pages[k]) {
                                max = k;
                                found = 1;
                                break;
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

found = 1;
break;
}
}

if (!found) {
    pos = j;
    break;
}

frame[pos] = pages[i];
faults++;

printf ("Pages %d : ", pages[i]);
for (i=0; i<f; i++) {
    printf ("%d", frame[i]);
}
printf ("\n");

printf ("faults = ");

int main () {
    int pages [50], n, f, i;
    printf ("Enter no. of pages: ");
    scanf ("%d", &n);
    printf ("Enter page references string");
    for (i=0; i<n; i++)
        scanf ("%d", &pages[i]);

    printf ("Enter no. of frames: ");
    scanf ("%d", &f);
}

```

FIFO (Pages, n, f)
LRU (Pages, n, f)
Optimal (Pages, n, f);
Return 0;

Q1

Enter no. of Pages: 4

Enter Page Reference String:

2

34

56

32

Enter no. of frames: 2

FIFO

Page 2 $\rightarrow$ 2 - 1

Page 34 $\rightarrow$ 2 34

Page 56 $\rightarrow$ 56 34

Page 32 $\rightarrow$ 56 32

Total Page Faults = 4

LRU

Page 2 $\rightarrow$ 2 - 1

Page 34 $\rightarrow$ 2 34

Page 56 $\rightarrow$ 56 34

Page 32 $\rightarrow$ 56 32

Total Page Faults = 4

Optimal

Page 2 $\rightarrow$ 2 - 1

Page 34 $\rightarrow$ 2 34

Page 56 $\rightarrow$ 56 34

Page 32 $\rightarrow$ 32 34

Total Page Faults = 4.